

How This Project Actually Works — Core Concepts

A plain-language walkthrough of the system, section by section. No file names, no function names — just the flow and the ideas behind it.

1. The Big Picture

Two machines talk directly to each other over the network — there's no middleman server relaying the file. One side offers a file, the other side receives it. Before any file content moves, they prove to each other that they share a secret, then they encrypt the file so that even if someone is watching the network traffic, they can't read it. The file is also broken into pieces so that a dropped connection doesn't mean starting over from zero.

That's the whole story at a high level: **connect** → **prove you belong** → **agree on a secret key** → **send encrypted pieces** → **verify nothing was corrupted or tampered with** → **reassemble**.

2. Proving You Belong (Authentication)

Before either side trusts the other, they need to prove they both know a shared secret code — without ever actually saying the code out loud over the network. If you just sent the secret code as plain text, anyone listening on the network could steal it and use it themselves.

The trick used here is called **challenge-response**: instead of sending the secret itself, one side sends a random, meaningless string of bytes (a "challenge") and asks the other side to prove they know the secret by mathematically combining the secret with that random challenge and sending back the result. Since the challenge is different every single time, an eavesdropper who records this exchange can't replay it later to pretend they're authenticated — the "proof" only works for that one random challenge, once.

The mathematical combination used is a **keyed hash** — you feed in both the secret and the challenge, and it spits out a fixed-length fingerprint that's practically impossible to produce without knowing the secret. The side asking for proof does the same computation itself and just checks whether the two fingerprints match.

One nuance worth understanding: in this system, this proof only flows in one direction. The side receiving the file is the one demanding proof (twice, for extra assurance) from the side sending it. The sender never demands the same proof back — it just trusts that if the receiver responds correctly at the end, it must be the real receiver. That's a meaningfully weaker guarantee than true two-way authentication, and it's worth being honest with yourself about that gap.

3. Turning a Password Into a Real Encryption Key

Humans pick weak, guessable passwords. Encryption algorithms need a strong, random-looking key. So there's a translation step in between.

That translation is done by a function whose whole purpose is to be **deliberately slow**. It takes the password and mixes it, over and over, tens of thousands of times, with a bit of randomness (a "salt") that's generated fresh for each file transfer. The slowness is the entire point: if an attacker steals the encrypted data and tries to guess the password by brute force, each guess costs them real computational time — multiplied across tens of thousands of rounds. That turns a fast, cheap attack into a slow, expensive one.

The salt is not a secret — it gets shared openly — but it guarantees that even if two people pick the exact same password, they end up with completely different final keys, because the randomness is different each time.

The output of this process is called the **session key**, and it's what protects this one particular file transfer.

4. One Key Isn't Enough — Deriving a Fresh Key Per Piece

The file isn't encrypted as one giant blob with a single key. Instead, it's cut into fixed-size pieces, and **each piece gets its own unique encryption key**, derived mathematically from the session key plus that piece's position number.

Why bother? Two reasons. First, it limits the damage if anything ever goes wrong with one piece's key — the other pieces aren't affected, because their keys are independent. Second, it makes it structurally impossible to accidentally reuse the exact same key-and-randomness combination twice, which is the one mistake that can completely break this style of encryption.

The technique for deriving these per-piece keys is the same keyed-hash idea from authentication: combine the session key with a label and the piece's position number, and out comes a brand-new key that looks nothing like the original.

5. Encrypting Each Piece

Each piece of the file is encrypted using a symmetric cipher — meaning the same key both locks and unlocks the data (as opposed to public-key systems, where locking and unlocking use different keys). Symmetric encryption is used here because it's dramatically faster than the public-key alternative, and speed matters when you're pushing megabytes of data through the pipe.

Alongside the unique key, each piece also gets its own random number used once (a **nonce**) that feeds into the encryption. Even with a unique key per piece, adding a fresh random nonce on top is extra insurance against ever reusing the exact same cryptographic inputs twice.

The encryption mode used here does something clever: it produces the scrambled data *and*, at the same time, a short **authentication tag** — basically a cryptographic seal that proves the data wasn't altered after it was encrypted. This means you get two properties from one operation: nobody can read the data without the key (**confidentiality**), and nobody can silently modify the data in transit without it being detected (**integrity**) — because tampering with even a single bit will make the tag fail to verify when the other side checks it.

6. Sending the Pieces and Handling Failure

Each encrypted piece is sent as its own self-contained transaction: connect, prove authentication again, send the piece, wait for acknowledgment, done. If the connection drops or times out partway through, only that one piece is retried — not the whole file. And several pieces can be sent at the same time, in parallel, using multiple simultaneous connections, which speeds up the overall transfer on a fast network.

If a piece keeps failing after a few attempts, the sending side pauses, asks the receiving side "which pieces do you already have?", and adjusts its plan based on the answer — sending only what's still missing. This can repeat a few times before the transfer is considered failed.

The tradeoff here is overhead: setting up a fresh connection (and re-proving authentication) for every single piece costs time compared to reusing one long-lived connection. The benefit is simplicity and resilience — each piece is independent, so failures are isolated and easy to reason about.

7. Remembering What's Already Been Received (Resumability)

The receiving side keeps a compact record — essentially a long string of yes/no flags, one per piece — tracking exactly which pieces of the file have already arrived and been safely written to disk. This is what makes "resume" possible: if the connection dies halfway through a large file, nothing is lost. The next time a transfer starts for that same file, the receiver reports back exactly which pieces it's missing, and the sender only sends those.

The final file on disk is actually pre-allocated to its full size up front, and pieces are written directly into their correct position within that file as they arrive — regardless of what order they show up in. This is important because pieces can legitimately arrive out of order when several are being sent in parallel.

Only once every single piece has been accounted for does the system treat the file as complete and rename the temporary file into its final, finished form.

8. Making Sure Nothing Got Corrupted or Tampered With (Integrity)

There are two separate layers of integrity checking happening here, at two different scales.

At the small scale, every individual piece has its own built-in tamper check (the authentication tag mentioned earlier) — if even one bit of a piece is altered in transit, decrypting it will fail loudly rather than silently producing corrupted data.

At the large scale, before the transfer even begins, the sender computes a single fingerprint — a **hash** — of the *entire* file as it exists on disk. This fingerprint is a fixed-length string that's essentially unique to that exact file content; changing even one byte anywhere in the file produces a completely different fingerprint. Once the receiver has reassembled every piece into a finished file, it independently recomputes that same fingerprint from what it actually received and compares it against the one the sender originally computed. If they match, you have strong end-to-end assurance that the file arrived byte-for-byte identical to the original — not just that each individual piece passed its own local check, but that the whole assembled result is correct.

9. Stopping Abuse and Replay Attacks

Two more defensive layers sit around all of this.

Replay protection: every transfer attempt carries a unique, randomly generated session identifier and a timestamp. The receiving side rejects anything with a timestamp too far in the past or future (guarding against someone recording an old exchange and trying to "replay" it later), and once a transfer has been marked fully complete, that same session identifier can never be reused again. This closes the door on someone capturing old network traffic and re-submitting it to trick the system into accepting an already-finished or forged transfer.

Rate limiting on failed logins: if a particular network address fails authentication too many times within a short window, that address gets temporarily blocked from even attempting authentication again for a while. This is a direct defense against brute-force guessing — without it, an attacker could just try secret codes rapidly until one worked; with it, guessing becomes prohibitively slow.

10. Putting It All Together

Read top to bottom, this is the full mental model of one transfer:

1. Two machines connect directly over the network.
2. They prove to each other (mostly one-directionally) that they know a shared secret, using random challenges instead of sending the secret itself.
3. A password gets deliberately, slowly stretched into a strong encryption key, using fresh randomness unique to this transfer.

4. That key branches into many independent per-piece keys, so no two pieces of the file ever share identical cryptographic material.
5. The file is split into pieces, each encrypted individually with its own key and its own one-time random value, producing both scrambled data and a tamper-evident seal.
6. Pieces are sent — potentially several at once — each as an independent, retryable transaction.
7. The receiving side tracks exactly which pieces have arrived, so an interrupted transfer can pick up later from exactly where it left off, not from scratch.
8. Every piece is checked for tampering as it arrives, and the whole reassembled file is checked one final time against a single fingerprint of the original.
9. Replay of old sessions and brute-force guessing are both actively defended against.

The core theme running through nearly every design choice here is **defense in depth**: no single mechanism is trusted alone. Authentication doesn't replace encryption. Per-piece tamper checks don't replace the whole-file fingerprint. Random nonces don't replace unique per-piece keys. Each layer catches what the layer next to it might miss.